Educational Component Library — Build Plan

Goal: Build and test the first template today using existing primitives so we are not starting from scratch. **Philosophy:** Borrow the plumbing. Build the educational layer.

The Layered Stack

```
YOUR EDUCATIONAL COMPONENTS     ← you build this
  HookHero, DefinitionCard,
  WorkedExampleCard, PracticeStack, etc.
        ↓ built on top of
SHADCN/UI PRIMITIVES            ← already exists, borrow it
  Accordion, Collapsible,
  Card, Dialog, Tabs, Badge,
  Separator, Tooltip, Progress
        ↓ built on top of
RADIX UI                        ← shadcn wraps this
  Accessibility, keyboard nav,
  ARIA, focus management
        ↓ plus
SUPPORTING LIBRARIES            ← already exists
  KaTeX (math rendering)
  Prism.js (code highlighting)
  Tailwind CSS (utility styling)
```

You are only writing the top layer.
Everything below it is already built, tested, and accessible.

Shadcn Primitive → Educational Component Mapping

This is the key insight. Each educational component is a thin wrapper around a shadcn primitive. You are adding educational design — content schema, capacity rules, pedagogical styling — not reinventing open/close logic or ARIA patterns.

```
Educational Component     Shadcn Primitive(s) Inside
──────────────────────────────────────────────────────
HookHero                  Card (no primitive needed — pure layout)
ExplanationBlock          Card + Typography
DefinitionCard            Card + Collapsible (for plain/formal toggle)
WorkedExampleCard         Card + Collapsible per step (step-reveal)
PracticeStack             Accordion (3 items, hint as CollapsibleContent)
DiagramBlock              Card + Dialog (for zoom overlay)
GlossaryRail              Card + ScrollArea (sticky sidebar)
ComparisonGrid            Table or CSS Grid (pure layout)
TimelineBlock             Separator + Card (pure layout)
ProcessSteps              Card + Collapsible per step
ReflectionPrompt          Card + Textarea (for write-in)
PitfallAlert              Alert (shadcn Alert component)
WhatNextBridge            Card (pure layout)
InterviewAnchor           Card (pure layout)
```

What You Install Once

```bash
```

```
# 1. shadcn/ui (if not already in the project)
npx shadcn@latest init

# 2. The primitives you need for the educational components
npx shadcn@latest add accordion
npx shadcn@latest add collapsible
npx shadcn@latest add card
npx shadcn@latest add dialog
npx shadcn@latest add alert
npx shadcn@latest add badge
npx shadcn@latest add separator
npx shadcn@latest add scroll-area
npx shadcn@latest add tabs
npx shadcn@latest add progress
npx shadcn@latest add tooltip
npx shadcn@latest add textarea

# 3. Math rendering
npm install katex

# 4. Code highlighting
npm install prismjs
```

That is the complete dependency list.
Everything else you write yourself on top of these.

The File Structure

```
components/
├── ui/                          ← shadcn primitives live here (auto-generated)
|   ├── accordion.tsx
|   ├── card.tsx
|   ├── collapsible.tsx
|   ├── dialog.tsx
|   ├── alert.tsx
|   └── ...
|
└── textbook/                    ← your educational components live here
    ├── index.ts                 ← exports all components
    ├── HookHero.tsx
    ├── ExplanationBlock.tsx
```

```
    ├── DefinitionCard.tsx
    ├── WorkedExampleCard.tsx
    ├── PracticeStack.tsx
    ├── DiagramBlock.tsx
    ├── GlossaryRail.tsx
    ├── ComparisonGrid.tsx
    ├── TimelineBlock.tsx
    ├── ProcessSteps.tsx
    ├── ReflectionPrompt.tsx
    ├── PitfallAlert.tsx
    ├── WhatNextBridge.tsx
    └── InterviewAnchor.tsx

templates/
└── guided-concept-path/
    ├── index.tsx                ← the template component
    ├── config.json              ← the content contract
    └── preset-default.css       ← the visual skin

lib/
└── textbook-types.ts            ← all TypeScript interfaces
```

Step 1 — Define the Types First (30 minutes)

Before writing any component, define the TypeScript interfaces. This is the content schema for every component — what the LLM fills in, what the renderer reads.

```typescript
```

```typescript
// lib/textbook-types.ts

export interface HookHeroContent {
  headline: string          // max 12 words
  body: string              // max 80 words
  anchor: string            // the felt need this creates
}

export interface ExplanationContent {
  body: string              // max 350 words
  emphasis: string[]        // max 3 key phrases to bold
}

export interface DefinitionContent {
  term: string
  formal: string            // max 80 words
  plain: string             // max 60 words
  etymology?: string        // optional
}

export interface WorkedStep {
  label: string             // max 12 words — why this step
  content: string           // max 80 words — what was done
}

export interface WorkedExampleContent {
  title: string
  setup: string             // max 60 words
  steps: WorkedStep[]       // max 6 steps
  conclusion: string        // max 40 words
}

export interface PracticeProblem {
  difficulty: 'warm' | 'medium' | 'cold'
  question: string          // max 100 words
  hint: string              // max 60 words
  writein_lines?: number    // 0-8, for print
}

export interface PracticeContent {
  problems: [PracticeProblem, PracticeProblem, PracticeProblem]
  // exactly 3 — enforced by the tuple type
}
```

```
export interface DiagramContent {
  svg_content: string
  caption: string           // max 60 words
  zoom_label?: string       // max 8 words
  alt_text: string          // accessibility
}

export interface GlossaryTerm {
  term: string
  definition: string        // max 30 words
  used_in?: string          // sentence where term first appears
}

export interface GlossaryContent {
  terms: GlossaryTerm[]     // max 8
}

export interface ComparisonCell {
  content: string           // max 20 words
}

export interface ComparisonContent {
  title: string
  columns: string[]         // max 4 column labels
  rows: Array<{
    axis: string            // max 8 words
    cells: string[]         // matches column count, max 20 words each
  }>                        // max 6 rows
}

export interface TimelineEvent {
  date: string
  label: string             // max 8 words
  body: string              // max 40 words
  weight: 'minor' | 'standard' | 'major'
}

export interface TimelineContent {
  title: string
  events: TimelineEvent[]   // max 12
}

export interface ProcessStepItem {
```

```
  number: number
  action: string            // max 15 words
  detail: string            // max 60 words
  input?: string
  output?: string
}

export interface ProcessContent {
  title: string
  steps: ProcessStepItem[]  // max 8
}

export interface ReflectionContent {
  prompt: string            // max 40 words
  type: 'connect' | 'predict' | 'question' | 'personal' | 'transfer'
  space?: number            // write-in lines
}

export interface PitfallContent {
  misconception: string     // max 20 words
  correction: string        // max 80 words
  example?: string          // max 40 words
}

export interface WhatNextContent {
  body: string              // max 50 words
  next: string              // max 15 words
}

export interface InterviewContent {
  prompt: string            // max 35 words
  audience: string          // max 10 words
}

// The section object — what the full section looks like
export interface SectionContent {
  section_id: string
  title: string
  subtitle?: string
  subject: string
  grade_band: string
  template_id: string
  hook: HookHeroContent
  explanation: ExplanationContent
```

```
  definition?: DefinitionContent
  worked_example?: WorkedExampleContent
  diagram?: DiagramContent
  pitfall?: PitfallContent
  practice: PracticeContent
  reflection?: ReflectionContent
  glossary?: GlossaryContent
  interview?: InterviewContent
  what_next: WhatNextContent
}
```

Step 2 — Build the Components (2-3 hours for core 5)

Build in this order — each one is useful on its own immediately:

Priority 1: PracticeStack

This is the most used component and the best test of shadcn integration. It uses Accordion for the three problems and Collapsible for the hints.

```tsx
```

```tsx
// components/textbook/PracticeStack.tsx

import { Accordion, AccordionContent,
         AccordionItem, AccordionTrigger } from "@/components/ui/accordion"
import { Badge } from "@/components/ui/badge"
import { Collapsible, CollapsibleContent,
         CollapsibleTrigger } from "@/components/ui/collapsible"
import { Button } from "@/components/ui/button"
import { PracticeContent } from "@/lib/textbook-types"

const difficultyConfig = {
  warm:   { label: 'Warm',   className: 'bg-emerald-50 text-emerald-700 border-emera
  medium: { label: 'Medium', className: 'bg-amber-50 text-amber-700 border-amber-200
  cold:   { label: 'Cold',   className: 'bg-blue-50 text-blue-700 border-blue-200' }
}

export function PracticeStack({ content }: { content: PracticeContent }) {
  return (
    <div className="space-y-1">
      <div className="text-xs font-semibold tracking-widest uppercase
                      text-muted-foreground pb-3 border-b mb-4">
        Practice Problems
      </div>
      <Accordion type="single" collapsible className="space-y-2">
        {content.problems.map((problem, i) => {
          const diff = difficultyConfig[problem.difficulty]
          return (
            <AccordionItem
              key={i}
              value={`problem-${i}`}
              className="border rounded-lg px-4 bg-card"
            >
              <AccordionTrigger className="hover:no-underline py-4">
                <div className="flex items-start gap-3 text-left">
                  <Badge variant="outline" className={diff.className}>
                    {diff.label}
                  </Badge>
                  <span className="text-sm leading-relaxed">
                    {problem.question}
                  </span>
                </div>
              </AccordionTrigger>
              <AccordionContent className="pb-4">
```

```
                <Collapsible>
                  <CollapsibleTrigger asChild>
                    <Button variant="ghost" size="sm"
                            className="text-muted-foreground text-xs h-7 px-2">
                      Show hint ↓
                    </Button>
                  </CollapsibleTrigger>
                  <CollapsibleContent>
                    <div className="mt-2 text-sm text-muted-foreground
                                    bg-muted/50 rounded-md p-3 leading-relaxed">
                      {problem.hint}
                    </div>
                  </CollapsibleContent>
                </Collapsible>
                {problem.writein_lines && problem.writein_lines > 0 && (
                  <div className="mt-3 space-y-3 print-only">
                    {Array.from({ length: problem.writein_lines }).map((_, j) => (
                      <div key={j} className="border-b border-muted h-8" />
                    ))}
                  </div>
                )}
              </AccordionContent>
            </AccordionItem>
          )
        })}
      </Accordion>
    </div>
  )
}
```

Priority 2: WorkedExampleCard

Uses Collapsible for step-reveal. Each step is hidden until the previous one has been revealed.

```tsx
```

```
// components/textbook/WorkedExampleCard.tsx

import { useState } from "react"
import { Card, CardContent, CardHeader, CardTitle }
  from "@/components/ui/card"
import { Button } from "@/components/ui/button"
import { Badge } from "@/components/ui/badge"
import { WorkedExampleContent } from "@/lib/textbook-types"

export function WorkedExampleCard({
  content,
  mode = 'step-reveal'
}: {
  content: WorkedExampleContent
  mode?: 'static' | 'step-reveal' | 'accordion'
}) {
  const [revealed, setRevealed] = useState(mode === 'static' ? 999 : 0)

  return (
    <Card className="border-l-4 border-l-violet-400">
      <CardHeader className="pb-3">
        <div className="text-xs font-semibold tracking-widest uppercase
                        text-violet-600 mb-1">
          Worked Example
        </div>
        <CardTitle className="text-base font-semibold">
          {content.title}
        </CardTitle>
        <p className="text-sm text-muted-foreground leading-relaxed">
          {content.setup}
        </p>
      </CardHeader>
      <CardContent className="space-y-3">
        {content.steps.map((step, i) => (
          <div
            key={i}
            className={`transition-all duration-300 ${
              i <= revealed ? 'opacity-100' : 'opacity-0 h-0 overflow-hidden'
            }`}
          >
            <div className="flex gap-3">
              <div className="flex-shrink-0 w-6 h-6 rounded-full
                              bg-violet-100 text-violet-700 text-xs
```

```
                              flex items-center justify-center font-bold">
                {i + 1}
              </div>
              <div>
                <div className="text-xs font-semibold text-violet-600 mb-1">
                  {step.label}
                </div>
                <div className="text-sm leading-relaxed text-foreground">
                  {step.content}
                </div>
              </div>
            </div>
          </div>
        ))}

        {mode === 'step-reveal' && revealed < content.steps.length - 1 && (
          <Button
            variant="outline"
            size="sm"
            onClick={() => setRevealed(r => r + 1)}
            className="mt-2 text-violet-600 border-violet-200"
          >
            Show next step →
          </Button>
        )}

        {revealed >= content.steps.length - 1 && content.conclusion && (
          <div className="mt-3 pt-3 border-t text-sm font-medium
                          text-foreground bg-violet-50 rounded-md p-3">
            {content.conclusion}
          </div>
        )}
      </CardContent>
    </Card>
  )
}
```

Priority 3: DefinitionCard

Uses Collapsible for the plain/formal toggle.

```
tsx
```

```tsx
// components/textbook/DefinitionCard.tsx

import { useState } from "react"
import { Card, CardContent } from "@/components/ui/card"
import { Button } from "@/components/ui/button"
import { DefinitionContent } from "@/lib/textbook-types"

export function DefinitionCard({ content }: { content: DefinitionContent }) {
  const [showFormal, setShowFormal] = useState(false)

  return (
    <Card className="border-l-4 border-l-purple-400 bg-purple-50/50">
      <CardContent className="pt-4">
        <div className="text-xs font-semibold tracking-widest uppercase
                        text-purple-600 mb-2">
          Definition
        </div>
        <div className="text-base font-semibold text-foreground mb-2">
          {content.term}
        </div>
        <p className="text-sm leading-relaxed text-foreground">
          {showFormal ? content.formal : content.plain}
        </p>
        {content.formal && content.plain && (
          <Button
            variant="ghost"
            size="sm"
            onClick={() => setShowFormal(f => !f)}
            className="mt-2 text-purple-600 text-xs h-7 px-2"
          >
            {showFormal ? 'Show plain language ↓' : 'Show formal definition ↑'}
          </Button>
        )}
        {content.etymology && (
          <p className="mt-2 text-xs text-muted-foreground italic">
            Etymology: {content.etymology}
          </p>
        )}
      </CardContent>
    </Card>
  )
}
```

Priority 4: HookHero

Pure layout — no shadcn primitive needed. This is the most visually distinctive component.

```tsx
// components/textbook/HookHero.tsx

import { HookHeroContent } from "@/lib/textbook-types"

export function HookHero({ content }: { content: HookHeroContent }) {
  return (
    <div className="relative rounded-xl bg-foreground text-background
                    p-8 overflow-hidden">
      {/* Decorative background character */}
      <div className="absolute right-4 top-0 text-[10rem] font-bold
                      text-white/[0.03] leading-none select-none
                      pointer-events-none">
        ?
      </div>
      <div className="relative">
        <div className="text-xs font-semibold tracking-widest uppercase
                        text-orange-300 mb-3">
          Intuition Hook
        </div>
        <h2 className="text-2xl font-bold leading-tight mb-3 max-w-lg">
          {content.headline}
        </h2>
        <p className="text-sm leading-relaxed text-white/70 max-w-lg">
          {content.body}
        </p>
      </div>
    </div>
  )
}
```

Priority 5: PitfallAlert

Uses the shadcn Alert component directly.

```tsx
```

```
// components/textbook/PitfallAlert.tsx

import { AlertTriangle } from "lucide-react"
import { Alert, AlertDescription, AlertTitle }
  from "@/components/ui/alert"
import { Collapsible, CollapsibleContent,
         CollapsibleTrigger } from "@/components/ui/collapsible"
import { Button } from "@/components/ui/button"
import { PitfallContent } from "@/lib/textbook-types"

export function PitfallAlert({ content }: { content: PitfallContent }) {
  return (
    <Alert className="border-orange-200 bg-orange-50">
      <AlertTriangle className="h-4 w-4 text-orange-600" />
      <AlertTitle className="text-orange-700 font-semibold text-sm">
        Common Pitfall — {content.misconception}
      </AlertTitle>
      <AlertDescription className="text-sm leading-relaxed mt-1">
        {content.correction}
      </AlertDescription>
      {content.example && (
        <Collapsible className="mt-2">
          <CollapsibleTrigger asChild>
            <Button variant="ghost" size="sm"
                    className="text-orange-600 text-xs h-6 px-2">
              See example →
            </Button>
          </CollapsibleTrigger>
          <CollapsibleContent>
            <div className="mt-2 text-xs text-muted-foreground
                            bg-white/60 rounded p-2 italic">
              {content.example}
            </div>
          </CollapsibleContent>
        </Collapsible>
      )}
    </Alert>
  )
}
```

Step 3 — Build the First Template (1 hour)

With five components built, you can assemble the Guided Concept Path template and test it with real content today.

```tsx
```

```tsx
// templates/guided-concept-path/index.tsx

import { HookHero } from "@/components/textbook/HookHero"
import { ExplanationBlock } from "@/components/textbook/ExplanationBlock"
import { DefinitionCard } from "@/components/textbook/DefinitionCard"
import { WorkedExampleCard } from "@/components/textbook/WorkedExampleCard"
import { PracticeStack } from "@/components/textbook/PracticeStack"
import { PitfallAlert } from "@/components/textbook/PitfallAlert"
import { WhatNextBridge } from "@/components/textbook/WhatNextBridge"
import { GlossaryRail } from "@/components/textbook/GlossaryRail"
import { SectionContent } from "@/lib/textbook-types"

export function GuidedConceptPath({
  section
}: {
  section: SectionContent
}) {
  return (
    <div className="max-w-4xl mx-auto">
      <div className="lg:grid lg:grid-cols-[1fr_220px] lg:gap-8
                      lg:items-start">

        {/* Main content column */}
        <div className="space-y-6">

          {/* Section header */}
          <div className="border-b pb-4">
            <div className="text-xs font-semibold tracking-widest
                            uppercase text-muted-foreground mb-1">
              {section.subject}
            </div>
            <h1 className="text-3xl font-bold tracking-tight">
              {section.title}
            </h1>
            {section.subtitle && (
              <p className="text-muted-foreground mt-1">
                {section.subtitle}
              </p>
            )}
          </div>

          {/* Hook — always first */}
          <HookHero content={section.hook} />
```

```
      {/* Explanation */}
      <ExplanationBlock content={section.explanation} />

      {/* Definition — if present */}
      {section.definition && (
        <DefinitionCard content={section.definition} />
      )}

      {/* Diagram — if present */}
      {section.diagram && (
        <DiagramBlock content={section.diagram} />
      )}

      {/* Worked example — if present */}
      {section.worked_example && (
        <WorkedExampleCard
          content={section.worked_example}
          mode="step-reveal"
        />
      )}

      {/* Pitfall — if present */}
      {section.pitfall && (
        <PitfallAlert content={section.pitfall} />
      )}

      {/* Practice — always present */}
      <PracticeStack content={section.practice} />

      {/* What's next — always last */}
      <WhatNextBridge content={section.what_next} />
    </div>

    {/* Glossary sidebar — desktop only */}
    {section.glossary && (
      <div className="hidden lg:block">
        <GlossaryRail
          content={section.glossary}
          className="sticky top-6"
        />
      </div>
    )}
  </div>
```

```
    </div>
  )
}
```

Step 4 — Test with Dummy Content (30 minutes)

Create a test page with hardcoded content that exercises all five components. This lets you see the template working before any LLM generation is hooked up.

```tsx
```

```
// app/test/page.tsx (or pages/test.tsx)

import { GuidedConceptPath } from "@/templates/guided-concept-path"
import { SectionContent } from "@/lib/textbook-types"

const testSection: SectionContent = {
  section_id: "test-01",
  title: "Why does calculus exist?",
  subtitle: "Two questions algebra cannot answer",
  subject: "Mathematics",
  grade_band: "secondary",
  template_id: "guided_concept_path_v1",

  hook: {
    headline: "How fast is something moving at this exact instant?",
    body: "You can measure where a ball is at two different moments. You can calcula
    anchor: "the gap between average speed and instantaneous speed"
  },

  explanation: {
    body: "This question — and one other equally frustrating one — forced mathematic
    emphasis: ["derivatives", "integrals", "opposites of each other"]
  },

  definition: {
    term: "Calculus",
    formal: "The mathematical study of continuous change, consisting of differential
    plain: "A branch of mathematics built to answer two questions: how fast is somet
    etymology: "From Latin calculus — small stone used for counting."
  },

  worked_example: {
    title: "Finding instantaneous speed from a position function",
    setup: "A ball is dropped from a 100-foot building. Its height at time t seconds
    steps: [
      {
        label: "Calculate position at t = 2",
        content: "h(2) = 100 - 16(4) = 100 - 64 = 36 feet. The ball is 36 feet high
      },
      {
        label: "Calculate average speed over one second",
        content: "h(3) = 100 - 16(9) = -44. Average speed = (-44 - 36) ÷ 1 = -80 ft/
      },
```

```
      {
        label: "Shrink the interval to see what happens",
        content: "Over 0.1 seconds: (h(2.1) − h(2)) ÷ 0.1 = −65.6 ft/s. Over 0.01 se
      },
      {
        label: "The limit gives the instantaneous speed",
        content: "As the interval approaches zero, the average speed approaches −64
      }
    ],
    conclusion: "The instantaneous speed is −64 ft/s. Calculus reaches this answer b
  },

  pitfall: {
    misconception: "Calculus is just faster algebra for the same problems",
    correction: "Calculus solves problems that algebra genuinely cannot. The instant
    example: "Trying to find instantaneous speed with only algebra requires dividing
  },

  practice: {
    problems: [
      {
        difficulty: "warm",
        question: "A car's position on a straight road is: at 1 PM it's at mile mark
        hint: "Average speed = distance ÷ time. Calculate separately for 1-2 PM and
        writein_lines: 4
      },
      {
        difficulty: "medium",
        question: "The height of a ball thrown upward is h(t) = 40t − 5t². Calculate
        hint: "Use (h(b) − h(a)) ÷ (b − a) for each interval. Notice how the average
        writein_lines: 5
      },
      {
        difficulty: "cold",
        question: "Without using calculus, explain why it is impossible to define th
        hint: "Think about what speed requires — distance divided by time. What happ
        writein_lines: 6
      }
    ]
  },

  glossary: {
    terms: [
      { term: "Derivative", definition: "The instantaneous rate of change of a funct
```

```
      { term: "Integral", definition: "The total accumulated change — the area under
      { term: "Limit", definition: "What a value approaches as a variable gets infin
      { term: "Instantaneous", definition: "At a single exact moment in time, as opp
    ]
  },

  what_next: {
    body: "Now that we understand why calculus exists, we need to develop the precis
    next: "Section 2 — Limits: The Foundation of Calculus"
  }
}

export default function TestPage() {
  return (
    <div className="min-h-screen bg-background p-6 md:p-10">
      <GuidedConceptPath section={testSection} />
    </div>
  )
}
```

Build Order for Today

```
Hour 1:
  ✓ Install shadcn primitives (10 min)
  ✓ Write textbook-types.ts (30 min)
  ✓ Build PracticeStack.tsx (20 min)

Hour 2:
  ✓ Build WorkedExampleCard.tsx (25 min)
  ✓ Build DefinitionCard.tsx (15 min)
  ✓ Build HookHero.tsx (10 min)
  ✓ Build PitfallAlert.tsx (10 min)

Hour 3:
  ✓ Build ExplanationBlock.tsx (10 min)
  ✓ Build WhatNextBridge.tsx (10 min)
  ✓ Assemble GuidedConceptPath template (20 min)
  ✓ Create test page with calculus dummy content (20 min)
  ✓ Open browser, see the template working (0 min — it just works)
```

By end of hour 3 you have a working template with real
interactive components, rendered from typed content,
with shadcn handling all the accessibility and animation plumbing.

What Comes Next (After Today)

```
Day 2:
  Build remaining components:
  GlossaryRail, DiagramBlock (with zoom Dialog),
  ComparisonGrid, TimelineBlock, ProcessSteps,
  ReflectionPrompt, InterviewAnchor

Day 3:
  Write config.json for GuidedConceptPath template
  Test LLM generation against the TypeScript types
  Validate content fits within capacity limits

Day 4:
  Build second template (Figure First or Compare and Apply)
  This goes faster because all components already exist

Week 2:
  Build the remaining templates
  Each new template is just a new assembly of existing components
  No new components needed for templates 2-5
```

The Key Insight for Speed

You are not building accordion logic. shadcn built it.
You are not building overlay/dialog logic. shadcn built it.
You are not building keyboard navigation. Radix UI built it.
You are not building ARIA patterns. Radix UI built it.

You are building the educational intelligence layer —
the content schemas, the visual styling that signals
pedagogical meaning, and the template assemblies.

That is 3–4 hours of work to have a working, testable template.
Not weeks. Today.

Build plan complete. Start with textbook-types.ts. Everything else follows from the types.